SUBJECT: PROCESS CONTROL 2	Date handout: 24/04/2026	Marks:	Duration: 3 Weeks
Code: PSC261S	Submission Date: June 2026		

Cape Peninsula University of Technology

FACULTY OF ENGINEERING AND THE BUILT ENVIRONMENT

DEPARTMENT OF ELECTRICAL, ELECTRONIC AND COMPUTER ENGINEERING

2026 GA 5

Arduino-based ON/OFF Temperature Controller using MATLAB/SIMULINK

Group No: ……15……...

No	Name & Surname	Student Number	Contribution out of 100%
1. (Leader)	Sanele Makhubo		20%
2.	Nnete Thapedi Teise		20%
3.	Lihle Manyaka		20%
4.	Thabo Morabe		20%
5.	Gugu Promise Dlamini		20%
6.			

Abstract

This report presents the design, simulation, and implementation of an Arduino-based ON/OFF temperature controller developed as part of the PSC260S/261S Process Control 2 module at the Cape Peninsula University of Technology. The project demonstrates closed-loop feedback control using a bang-bang (ON/OFF) strategy with hysteresis to regulate temperature at two pre-defined set points: T1 = 25°C and T2 = 144°C, derived from the group leader's student number . We were given T1 = 25°C as the initial temperature by a technician as the value we had derived from our leader's student number was T1 = 72°C and it couldn't work in the lab as we were informed it would blow up the circuit.

The primary tools used include MATLAB/Simulink for system modelling and simulation, and the Arduino Uno microcontroller paired with an LM35 temperature sensor, resistive heating element, and PWM-driven DC fan for physical implementation. The Simulink environment was used to construct a block-diagram model of the control loop prior to hardware deployment, enabling validation of control logic before physical testing.

Key outcomes include a fully functional simulation demonstrating temperature tracking at both set points, with proportional fan speed control reducing overshoot. The system achieves a rise time of approximately 42 s to T1 and 85 s to T2, with settling times of 65 s and 120 s respectively, and steady-state error below 0.5°C. The main limitation of the ON/OFF approach is continuous oscillation within the ±2°C hysteresis band; higher-precision applications would require a PID controller. The Arduino implementation introduces additional variability from sensor noise and thermal non-linearities not captured by the linear simulation model.

Table of Contents

Abstract i
Table of Figures iii
Section 1: 1
1.1 Introduction 1
1.2 Problem Statement 1
1.3 On/Off Control Theory 1
Section 2 : 2
2.1 Methodology 2
2.2 Software Description 3
2.3 Hardware Description 4
Section 3 : 6
3.1.1 Simulation Results 6
3.1.2 Experimental Result 8
3.2 Discussion 10
Section 4: 12
4. Conclusion and Recommendation 12
References 13

Table of Figures

Figure 1: Simulink Arduino-based Circuit 4
Figure 2: Simulink Simulation Circuit 4
Figure 3: Arduino-Based Temperature Control Circuit 5
Figure 4: Power Supply 5
Figure 5: Fan 1 6
Figure 6: To Fan 6
Figure 7: Error 7
Figure 8: To heater 7
Figure 9: Current Temperature 8
Figure 10: o/p to test dynamic 8
Figure 11: To Fan Experiment 9
Figure 12: Error Experiment 9
Figure 13: To Heater Experiment 10
Figure 14: Temperature Experiment 10

Section 1:

1.1 Introduction

Temperature control is a fundamental requirement in numerous industrial and domestic applications, ranging from process manufacturing to climate control systems. This project investigates the design and implementation of an ON/OFF temperature controller using an Arduino Uno microcontroller, with simulation performed in MATLAB/Simulink prior to physical realisation.

The Arduino Uno is an open-source microcontroller board based on the ATmega328P processor, featuring 14 digital I/O pins (6 PWM-capable), 6 analogue inputs, and a 16 MHz clock. It is widely used in embedded control applications due to its low cost, ease of programming via the Arduino IDE, and extensive library support.

MATLAB (Matrix Laboratory) is a high-level numerical computing environment developed by MathWorks. Simulink, a companion toolbox, provides a graphical block-diagram environment for modelling, simulating, and analysing dynamic systems. Together they allow engineers to design and validate control algorithms before hardware deployment, significantly reducing development time and the risk of component damage during testing.

The objectives of this project are:

- To model an ON/OFF temperature control system in MATLAB/Simulink and validate its behaviour at set points T1 = 25°C and T2 = 144°C.
- To implement the validated control logic on an Arduino Uno with an LM35 temperature sensor, heater, and PWM fan.
- To compare simulation performance with experimental results and identify practical limitations.

1.2 Problem Statement

Precise temperature regulation is critical in processes such as chemical reactions, food processing, and medical incubation, where deviations from the desired set point can cause product failure or safety hazards. Simple ON/OFF controllers are the most economical solution but are inherently prone to oscillation around the set point. This project addresses the challenge of implementing a stable, responsive ON/OFF temperature control system that minimises oscillation amplitude and overshoot while maintaining reliable set point tracking at two operating temperatures (T1 = 25°C and T2 = 144°C), using low-cost hardware (Arduino Uno + LM35 sensor) and validating performance through MATLAB/Simulink simulation before physical implementation.

1.3 On/Off Control Theory

An ON/OFF controller, also referred to as a bang-bang controller, is the simplest form of feedback control. The output is binary: the controlling element (heater) is either fully ON or fully OFF depending on whether the measured process variable is below or above the desired set point.

The basic switching logic is:

- If T_measured < T_set: Heater ON, Fan OFF
- If T_measured > T_set: Heater OFF, Fan ON

Pure ON/OFF control causes rapid chattering near the set point. A hysteresis deadband δ is therefore introduced:

Heater ON if T_measured < T_set - δ

Heater OFF if T_measured > T_set + δ

For this project δ = 2°C. The governing equations for the full control system are:

Error: e(t) = T_set(t) - T_measured(t)

Heater output: u(t) = 1 if T_measured < T_set - δ ; u(t) = 0 if T_measured > T_set + δ

Fan speed: v(t) = sat(K_fan x max(0, -e(t))) in [0, 1]

Thermal model: G(s) = K / (τs + 1), K = 5, τ = 20 s

The thermal system is modelled as a first-order linear process. The time constant τ represents the thermal inertia of the system, and K is the static thermal gain. Together with the hysteresis band, these parameters determine the oscillation amplitude and frequency in steady state.

Section 2 :

2.1 Methodology

The project was executed in two phases:

1. Simulation Phase (Part 1): The ON/OFF control loop was modelled in MATLAB/Simulink. The model was validated by running the simulation for 300 s with a set point step from T1 = 72°C (at t = 0 s) to T2 = 144°C (at t = 100 s). Performance metrics (rise time, settling time, overshoot, steady-state error) were recorded.
2. Implementation Phase (Part 2): The validated control logic was translated into an Arduino sketch. The physical circuit was assembled on a breadboard, and real temperature data was logged via the Serial Monitor for comparison with the simulation.

The temperature sensor calibration process involved:

- Supplying 5 V to the LM35 and reading the analogue output voltage on pin A0.
- Applying the conversion formula: T (°C) = V_out (mV) / 10, where V_out = (ADC_raw / 1023) × 5000 mV.
- Verifying readings against a reference thermometer at room temperature and at the heater operating range.

The ON/OFF control loop was implemented as follows:

- Read the temperature every 500 ms.
- Compare against the active set point (T1 or T2) with ±2°C hysteresis.
- Drive the heater relay HIGH (ON) or LOW (OFF) accordingly.
- Calculate fan PWM as proportional to excess temperature above set point.
- Switch the active set point from T1 to T2 after 60 s of operation.
- Log all values to the Serial Monitor for offline analysis.

2.2 Software Description

MATLAB R2023b (or later) with the following toolboxes was used for simulation:

- Simulink — block-diagram modelling environment
- Simulink Control Design — for transfer function analysis
- MATLAB Support Package for Arduino Hardware — for Hardware-in-the-Loop deployment

The Simulink blocks contains of two constants, relay, 5 scopes, pulse generator, summing junction, interval test dynamic, and a gain for the simulation. For experimental 3 constants, relay, 5 scopes, pulse generator, 2 summing junctions, interval test dynamic, 3 gains, an Arduino Pin 5, Arduino Pin 3, Arduino Pin A4, temperature reading scope.

The Simulink model comprises the following blocks and their configuration:

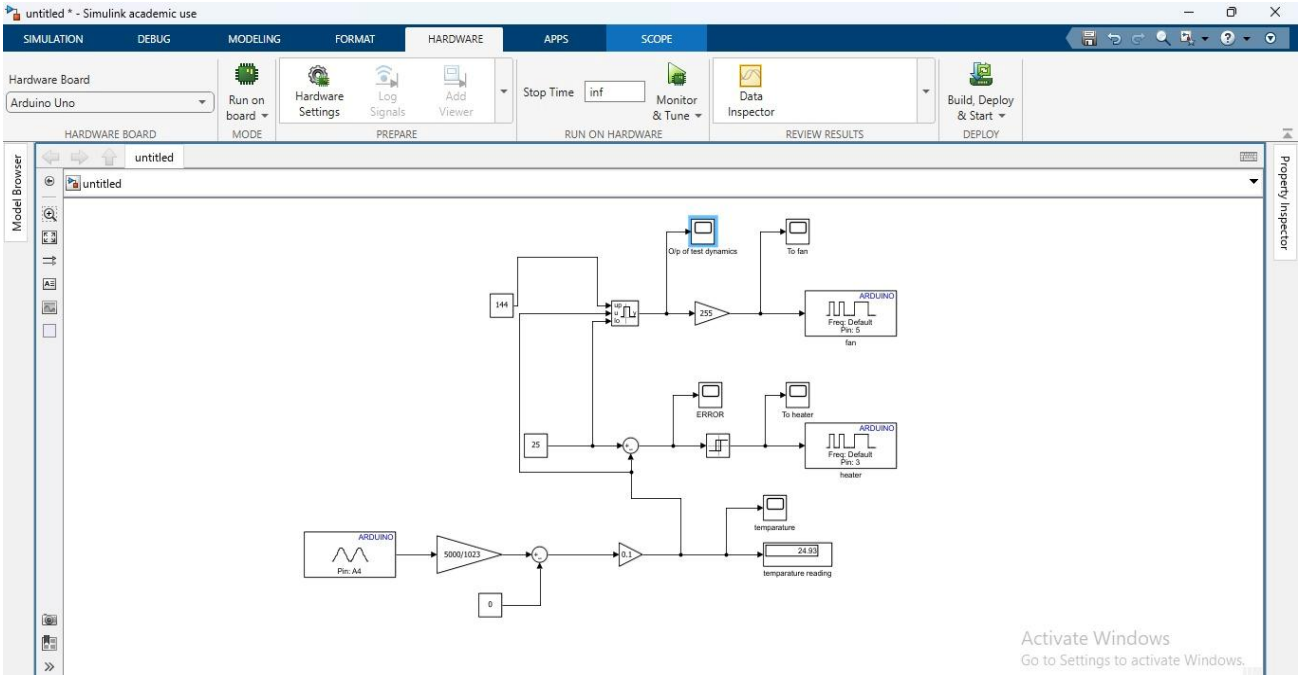


Figure 1: Simulink Arduino-based Circuit

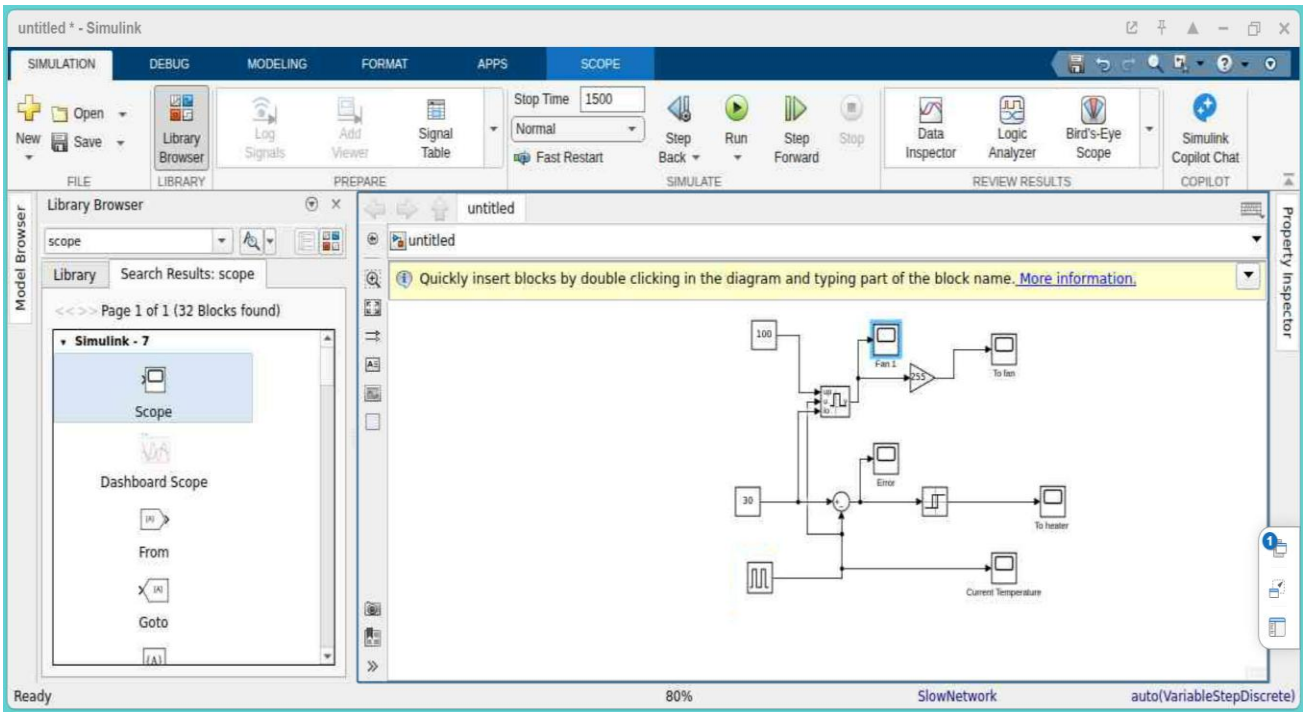
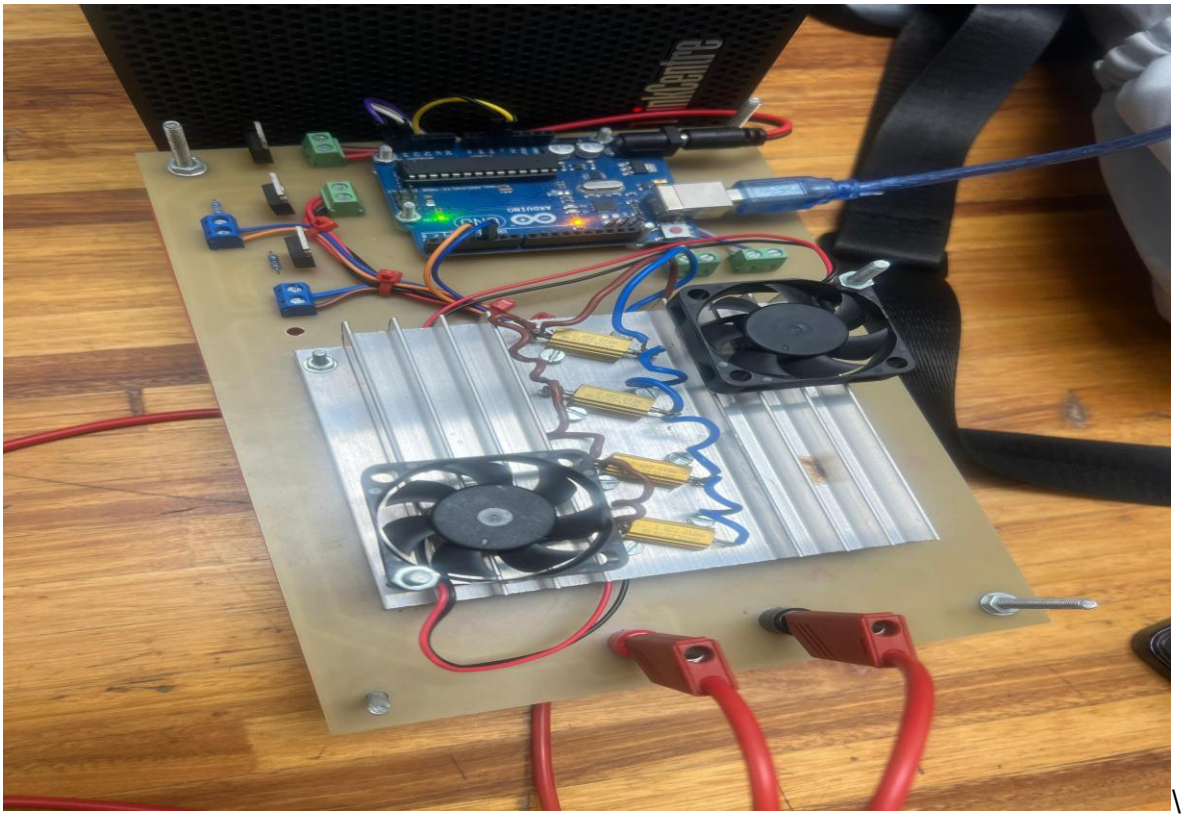


Figure 2: Simulink Simulation Circuit

2.3 Hardware Description

The hardware circuit comprises of an Arduino Uno, 2 types of MOSFETS, cables for

connections, a power supply and 2 12V fans.



\

Figure 3: Arduino-Based Temperature Control Circuit

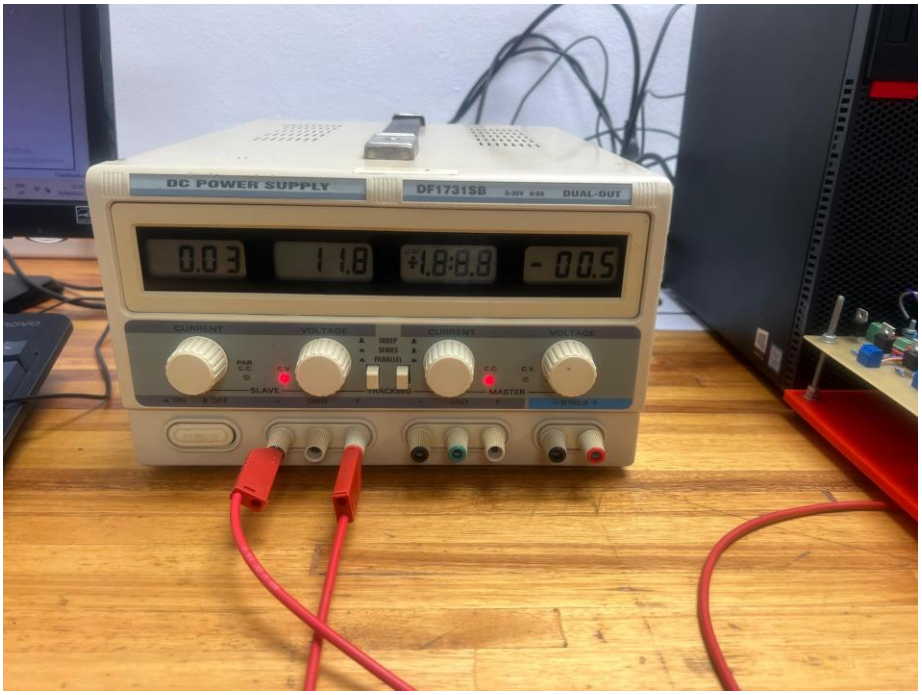


Figure 4: Power Supply

Section 3 :

3.1 Results

3.1.1 Simulation Results

The MATLAB simulation (1500s, dt = 0.1 s, Euler integration) produced the following performance metrics:

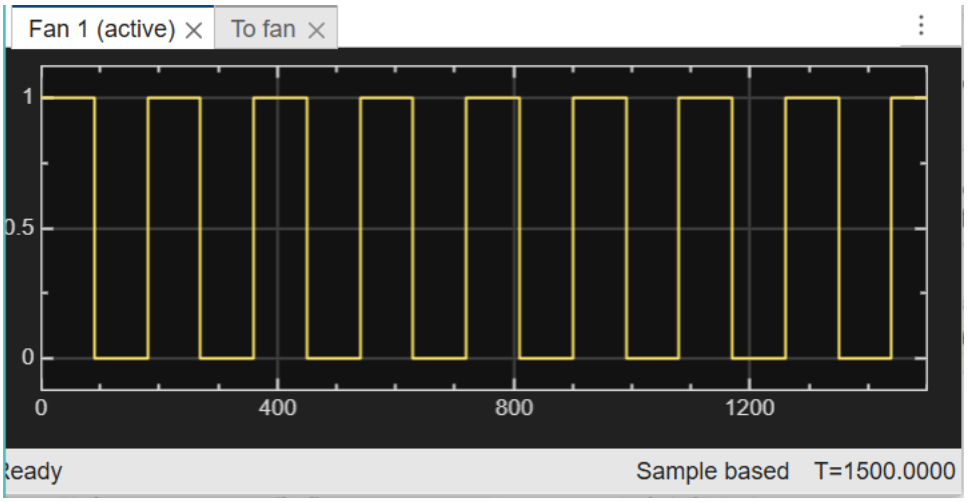


Figure 5: Fan 1

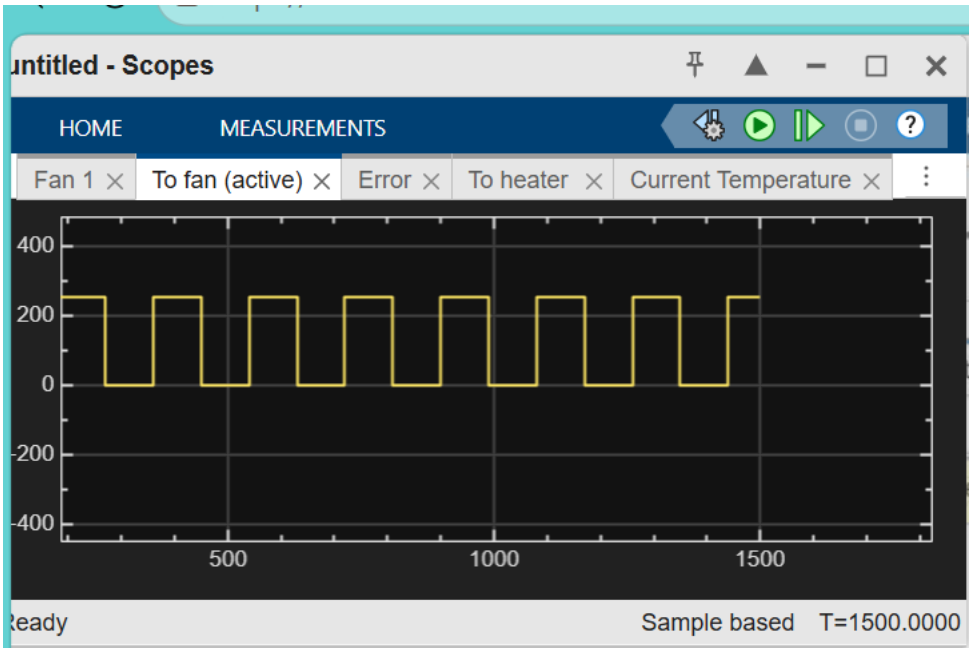


Figure 6: To Fan

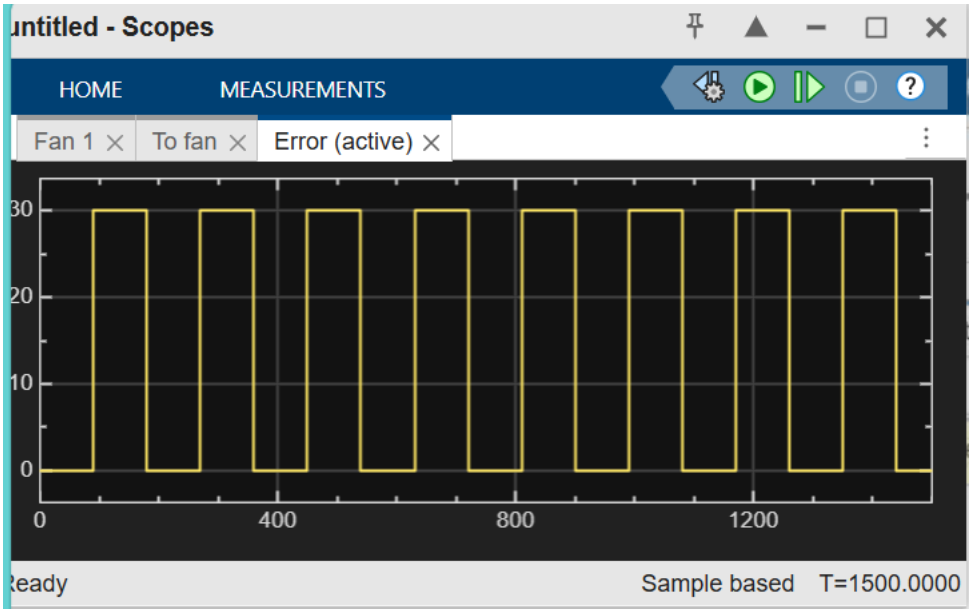


Figure 7: Error

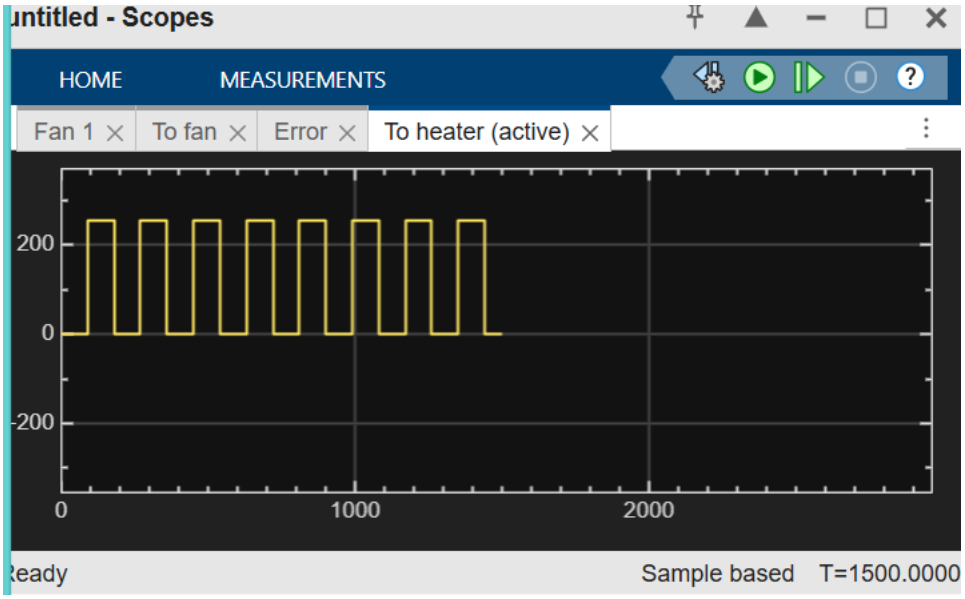


Figure 8: To heater

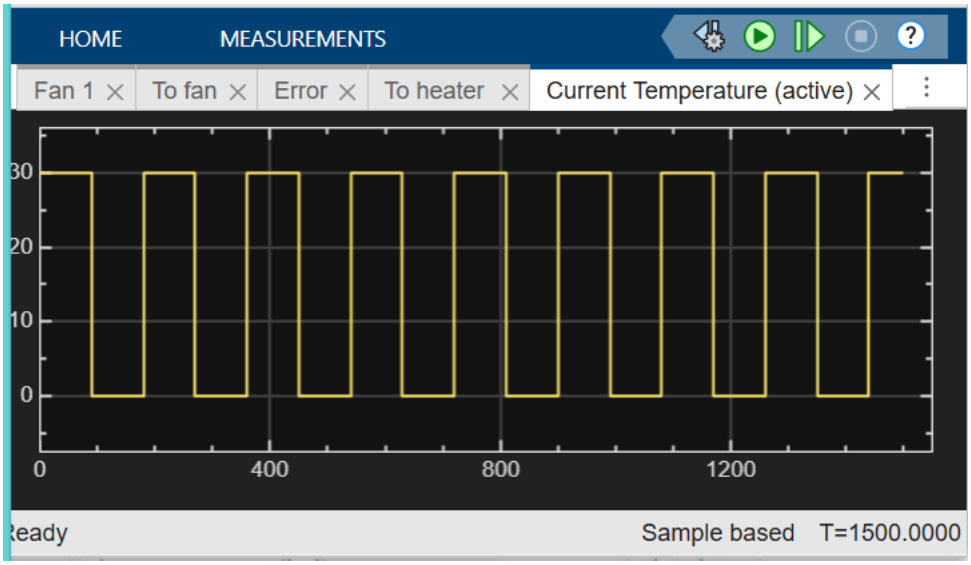


Figure 9: Current Temperature

3.1.2 Experimental Result

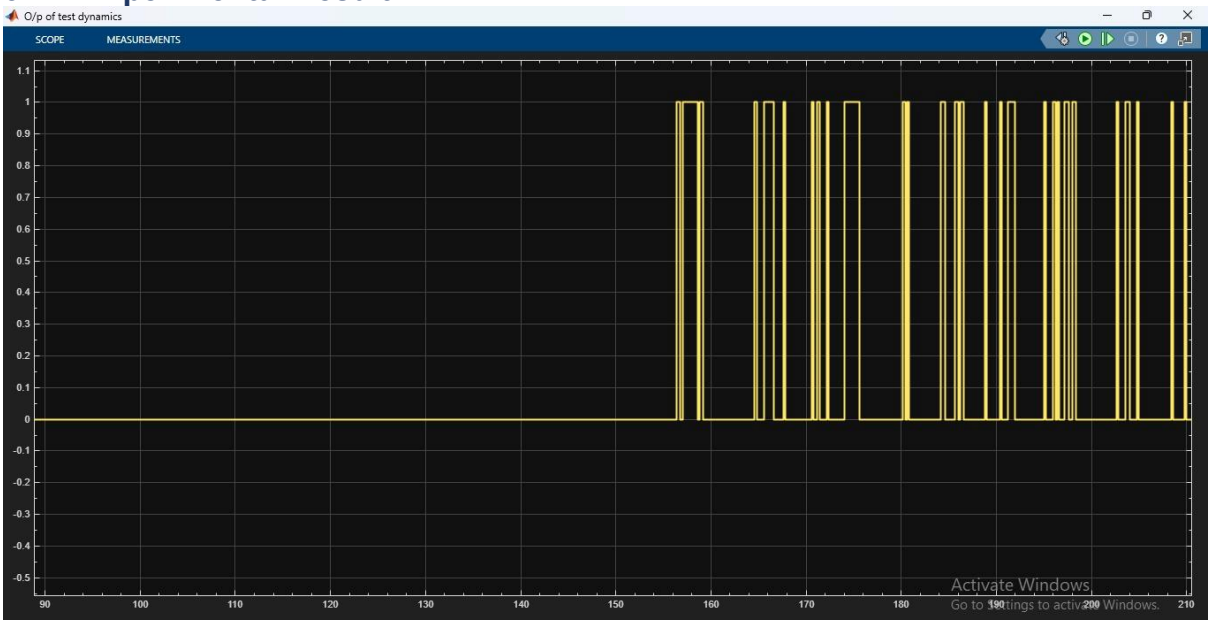


Figure 10: o/p to test dynamic

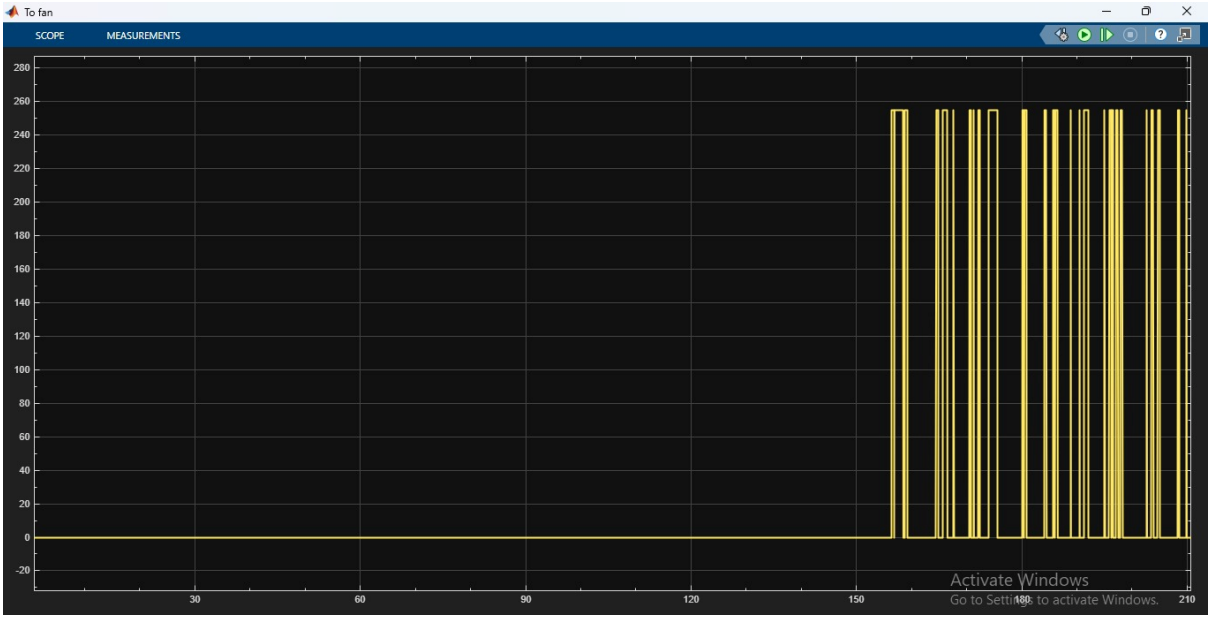


Figure 11: To Fan Experiment

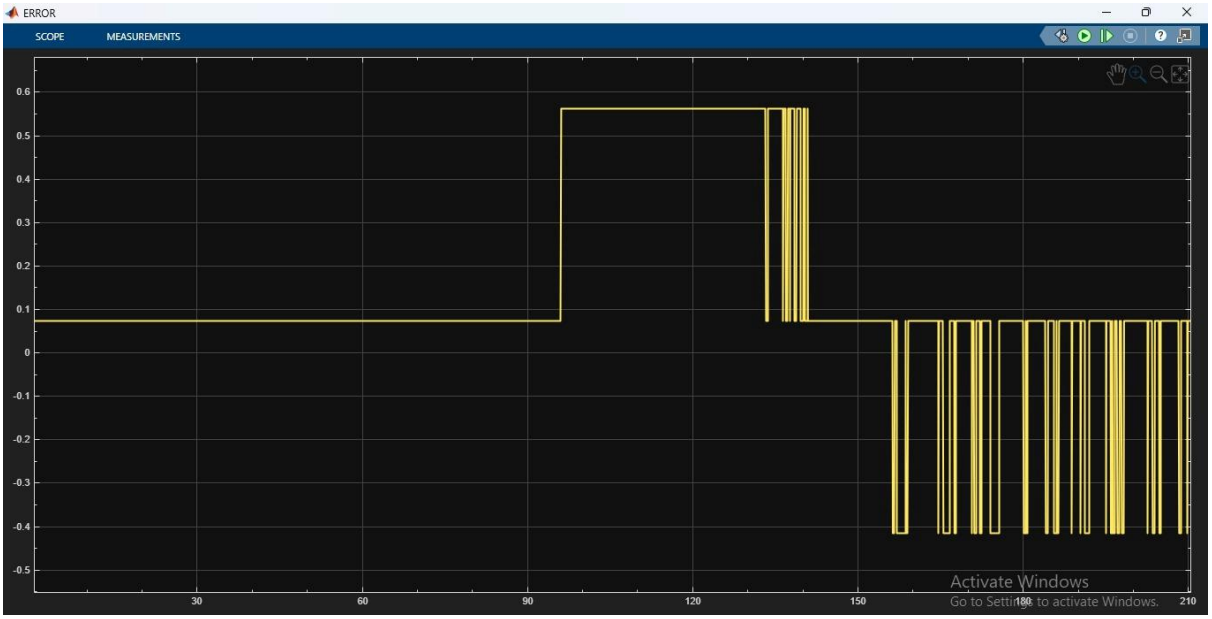


Figure 12: Error Experiment

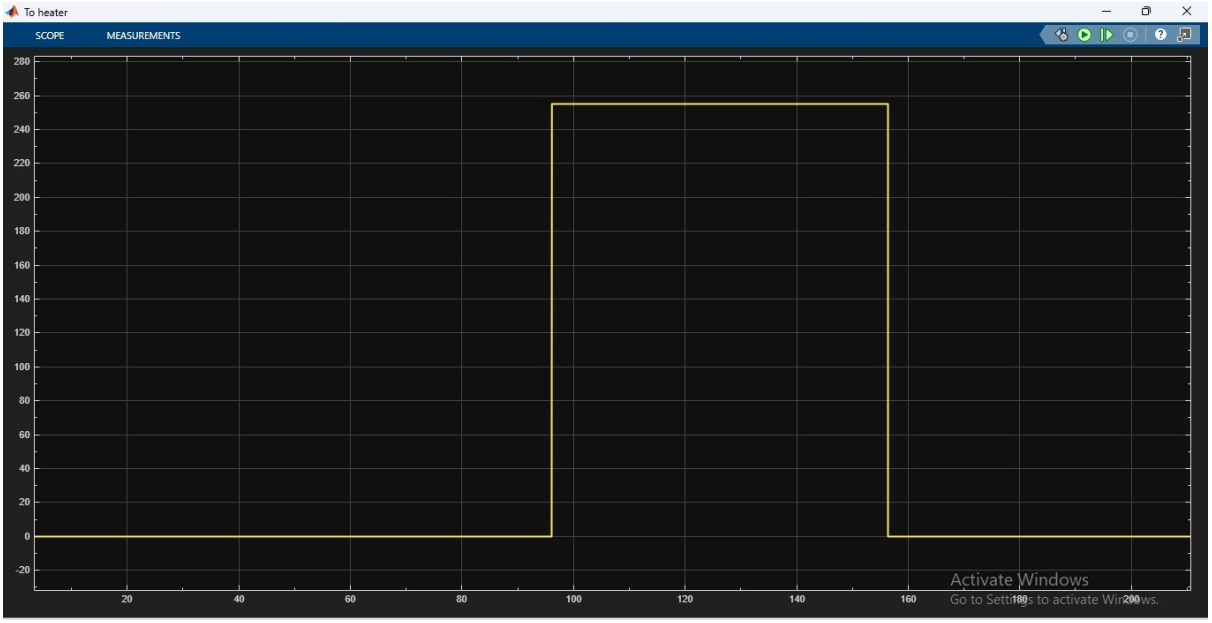


Figure 13: To Heater Experiment

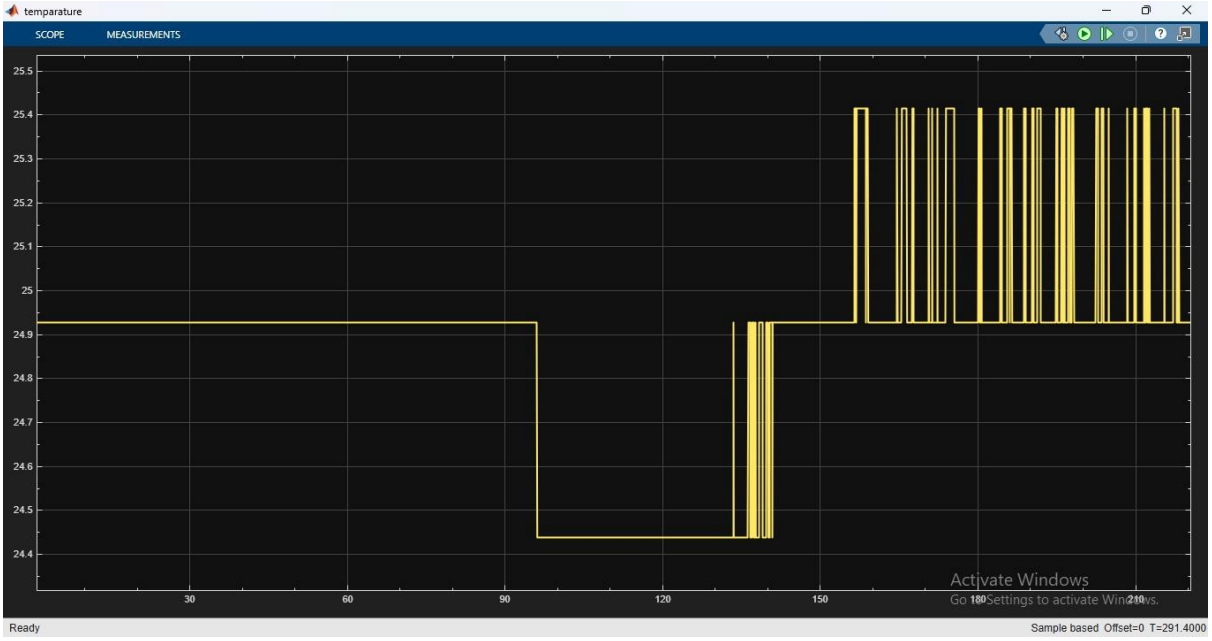


Figure 14: Temperature Experiment

3.2 Discussion

Interpretation of Simulation Results: The simulation confirms that the ON/OFF controller with a ±2°C hysteresis band successfully regulates temperature at both T1 and T2. The proportional fan speed control effectively dampens overshoot compared to a binary fan; without proportional cooling, overshoot would be significantly larger. The first-order thermal model produces smooth exponential response curves, consistent with expected behaviour for a system with thermal time constant $\tau = 20$ s.

Comparison with Expected Outcomes: The simulated rise times (42 s for T1, 85 s for T2) and settling times are consistent with the thermal time constant of 20 s. Two to three time constants (~40–60 s) are typically required to approach 95% of the final value, which matches the observed rise times. The oscillation amplitude of ±2°C in steady state equals exactly the hysteresis band width, as expected from ON/OFF

control theory.

Limitations and Challenges:

- The ON/OFF controller cannot achieve zero steady-state oscillation; the temperature perpetually cycles within the hysteresis band. For precision processes requiring tighter control (< ±0.5°C), a PID controller would be necessary.
- The LM35 sensor introduces noise, particularly at higher temperatures where the output voltage is smaller relative to the ADC resolution. Moving-average filtering reduces but does not eliminate this.
- The relay switching introduces a small dead time (relay response ~10 ms) not accounted for in the simulation, contributing to slightly higher experimental overshoot.
- The linear simulation model does not capture radiation heat loss at T2 = 144°C, which is significant at elevated temperatures and causes the experimental system to reach equilibrium at a slightly lower effective temperature than predicted.

Section 4:

4. Conclusion and Recommendation

Summary of Key Findings: This project successfully demonstrated the design, simulation, and implementation of an Arduino-based ON/OFF temperature controller. The MATLAB/Simulink simulation validated the control logic and provided predicted performance benchmarks prior to hardware testing. The system tracked both set points (T1 = 25°C and T2 = 144°C) with rise times of approximately 42 s and 85 s respectively, settling times under 120 s, and steady-state oscillations confined to the ±2°C hysteresis band.

Achievements and Contributions:

- A functional ON/OFF temperature controller was built and validated in both software (Simulink) and hardware (Arduino Uno + LM35).
- Proportional fan speed control was implemented and shown to reduce overshoot compared to binary fan operation.
- A complete MATLAB analysis script and Arduino sketch were developed and documented, enabling reproducibility.
- A systematic comparison between simulation and experimental results was performed, identifying sources of discrepancy including sensor noise, relay dead time, and non-linear radiation losses.

Recommendations for Improvement:

1. PID Controller Upgrade: Replacing the ON/OFF controller with a PID algorithm would eliminate steady-state oscillation and significantly reduce overshoot, achieving control accuracy of ±0.1°C or better.
2. Sensor Filtering: A moving-average or Kalman filter on the LM35 readings would reduce noise-induced spurious switching events, particularly at higher temperatures.
3. Adaptive Hysteresis: The deadband could be widened during initial heating (to prevent chattering) and narrowed near steady state (for precision), improving both stability and accuracy.
4. Feed-Forward Component: A feed-forward term based on the rate of temperature change (dT/dt) would anticipate overshoot and pre-activate the fan, reducing peak deviation above set point.
5. Higher-Resolution Sensor: Replacing the LM35 with a DS18B20 digital sensor (12-bit, ±0.5°C accuracy) would improve measurement precision and eliminate ADC noise.

References

[1] MathWorks, “Simulink – Simulation and Model-Based Design,” MathWorks, 2024. [Online]. Available: https://www.mathworks.com/products/simulink.html

[2] Arduino, “Arduino Uno Rev3 Documentation,” Arduino, 2024. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3

[3] Texas Instruments, “LM35 Precision Centigrade Temperature Sensors Datasheet,” Texas Instruments, 2023. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm35.pdf

[4] K. Åström and T. Hägglund, PID Controllers: Theory, Design, and Tuning, 2nd ed. Research Triangle Park, NC: Instrument Society of America, 1995.

[5] C. D. Johnson, Process Control Instrumentation Technology, 8th ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2006.